



SmartShelf ESL Open API

V1.0.0



Integration reference for electronic shelf label operations
— products, pricing, bulk loading and scheduled display.

Document history

Revision	Release date	Changes / notes
V1.0.0	2026.08.14	First release. Product, bulk, device and scheduling APIs for Minew ESL.

Contents

Part 1. Before you begin

Part 2. Authentication

2.1 Log in	
------------	-------

Part 3. Reference data

3.1 Query category list	
3.2 Query sub-category list	
3.3 Query label list	
3.4 Query template list	

Part 4. Product API

4.1 Add product	
4.2 Add product with labels	
4.3 Modify product	
4.4 Modify product with labels	
4.5 Query product list	
4.6 Query product by product code	
4.7 Query product by identifier	
4.8 Delete product	

Part 5. Bulk API

5.1 Add products in batch	
5.2 Modify products and prices in batch	

Part 6. Label API

6.1 Query label by MAC address	
6.2 Query label by identifier	
6.3 Redraw label	

Part 7. Scheduling API

7.1 Add schedule	
7.2 Add schedule from an existing binding	
7.3 Query schedule list	
7.4 Query schedule and execution history	
7.5 Query upcoming schedules	
7.6 Query active schedules	
7.7 Query schedules for a label	
7.8 Modify schedule	
7.9 Run schedule immediately	
7.10 Stop schedule	
7.11 Delete schedule	
7.12 Query priority list	

Appendix A. Response envelope

Appendix B. Status codes

Appendix C. Troubleshooting

Note

Read before working through the parts that follow.

- All URLs in this document are relative to the base address of your SmartShelf deployment. Replace `https://your-server` with the address supplied to you — for a local installation this is normally `http://localhost:5000`.
- Every request except **Log in** requires the bearer token returned by **Part 2.1**, sent as the header `Authorization: Bearer <token>`.
- Every product operation is scoped to a single store. Product codes are unique *per store* and not globally, so `storeId` is required wherever it appears.
- Prices reach the electronic shelf labels as part of the call that changes them. There is no separate synchronise or publish endpoint to call afterwards.

Part 1. Before you begin

This part explains the conventions used throughout the document. It contains no endpoints. Read it once before working through the parts that follow.

Base address

Every URL is shown relative to your deployment's base address. Substitute the address supplied to you wherever `https://your-server` appears.

Authentication

SmartShelf uses bearer-token authentication. Call **Part 2.1 Log in** once, then send the returned token on every subsequent request as the header `Authorization: Bearer <token>`. The token also identifies the store the signed-in user belongs to.

Roles

Read operations are available to every signed-in user. Operations that create, change or delete data require the **Admin**, **Manager** or **Operator** role; a user with the **Viewer** role receives `403`. Where an endpoint is more restricted than this, its description says so.

Response envelope

Every endpoint returns the same envelope. Read `success` first; the payload is always under `result`. The full shape is given in **Appendix A**.

Two conditions for a price to reach a label

First, the store must be linked to a Minew cloud store. Second, a label must be bound to the product with a template. Sending `eslAssignments` when creating or updating a product satisfies the second condition automatically. **Appendix C** covers what happens when either is missing.

Part 2. Authentication

Obtain the bearer token used by every other endpoint in this document.

2.1 Log in

URL	<code>https://your-server/api/auth/login</code>
Request method	POST
Content-type	application/json;charset=utf-8
Authorisation	None. This is the only endpoint that does not require a token.
Description	Exchanges an employee identifier and password for a bearer token. The response also carries the user's store, which supplies the <code>storeId</code> used throughout the rest of this document.

- `userName` is the **Employee ID**, not the user's email address. Supplying the email address returns `401`.

REQUEST PARAMETERS

Key	Position	Type	Required	Example	Description
<code>userName</code>	Body	String	Yes	<code>"ADMIN01"</code>	Employee identifier.
<code>password</code>	Body	String	Yes	<code>"....."</code>	Account password, sent in plain text over TLS.

REQUEST DEMO

```
{
  "userName": "ADMIN01",
  "password": "....."
}
```

EXPLANATION FOR RETURN CODE

Status code	Description
<code>200</code>	Success. A token is returned.
<code>401</code>	Invalid credentials, or the employee identifier does not exist.
<code>500</code>	Unexpected server error.

SAMPLE RETURN RESULT

```
{
  "success": true,
  "message": "Login successful",
  "responsCode": 200,
  "result": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "user": {
      "id": 42,
      "employeeId": "ADMIN01",
      "firstName": "Nimal",
      "lastName": "Perera",
      "email": "admin01@retailit.lk",
      "storeId": 3,
      "store": {
        "id": 3,
        "storeName": "Colombo City Centre",
        "storeCode": "CCC-01",
        "minewStoreId": "2087891877587062784"
      }
    }
  }
}
```

Part 3. Reference data

Read-only lookups that supply the identifiers used when building product and label payloads. Call these once and cache the results; they change rarely.

3.1 Query category list

URL	<code>https://your-server/api/products/all-categories</code>
Request method	GET
Content-type	—
Authorisation	Bearer token.
Description	Product categories defined for a store. Supplies the values used as <code>categoryId</code> , or as <code>categoryName</code> in bulk payloads.

REQUEST PARAMETERS

Key	Position	Type	Required	Example	Description
<code>storeId</code>	Query	Long	Yes	<code>3</code>	Store to list categories for.
<code>pageNumber</code>	Query	Int	No	<code>1</code>	Page number. Defaults to 1.
<code>pageSize</code>	Query	Int	No	<code>100</code>	Rows per page. Defaults to 10, maximum 100.
<code>searchTerm</code>	Query	String	No	<code>"Bev"</code>	Filters on category name.

REQUEST DEMO

```
GET /api/products/all-categories?storeId=3&pageNumber=1&pageSize=100
```

EXPLANATION FOR RETURN CODE

Status code	Description
<code>200</code>	Success.
<code>401</code>	Missing, expired or invalid bearer token.
<code>404</code>	No matching record for the supplied identifier and store.
<code>500</code>	Unexpected server error.

SAMPLE RETURN RESULT

```
{
  "success": true,
  "message": "Categories retrieved successfully.",
  "responsCode": 200,
  "result": {
    "items": [
      { "id": 3, "categoryCode": "BEV", "categoryName": "Beverages", "isActive": true, "storeId": 3 },
      { "id": 4, "categoryCode": "BAK", "categoryName": "Bakery", "isActive": true, "storeId": 3 }
    ],
    "totalCount": 2,
    "pageNumber": 1,
    "pageSize": 100,
    "totalPages": 1
  }
}
```

3.2 Query sub-category list

URL	<code>https://your-server/api/products/active-subcategories</code>
Request method	GET
Content-type	—
Authorisation	Bearer token.
Description	Active sub-categories, optionally narrowed to one category. A sub-category is always subordinate to a category; supplying one that belongs to a different category is rejected when the product is saved.

REQUEST PARAMETERS

Key	Position	Type	Required	Example	Description
<code>storeId</code>	Query	Long	Yes	<code>3</code>	Store to list sub-categories for.
<code>categoryId</code>	Query	Long	No	<code>3</code>	Restricts the result to one category.
<code>pageNumber</code>	Query	Int	No	<code>1</code>	Page number.
<code>pageSize</code>	Query	Int	No	<code>100</code>	Rows per page.

REQUEST DEMO

```
GET /api/products/active-subcategories?storeId=3&categoryId=3
```

EXPLANATION FOR RETURN CODE

Status code	Description
200	Success.
401	Missing, expired or invalid bearer token.
404	No matching record for the supplied identifier and store.
500	Unexpected server error.

SAMPLE RETURN RESULT

```
{
  "success": true,
  "message": "Sub-categories retrieved successfully.",
  "responsCode": 200,
  "result": {
    "items": [
      { "id": 4, "subCategoryName": "Tea & Coffee", "categoryId": 3, "isActive": true, "storeId":
3 }
    ],
    "totalCount": 1,
    "pageNumber": 1,
    "pageSize": 100,
    "totalPages": 1
  }
}
```

3.3 Query label list

URL	<code>https://your-server/api/device/devices/local</code>
Request method	GET
Content-type	—
Authorisation	Bearer token.
Description	Every active electronic shelf label held by SmartShelf. The <code>id</code> values returned here are used as <code>deviceId</code> when binding a product to a label.

REQUEST PARAMETERS

This endpoint takes no parameters.

REQUEST DEMO

```
GET /api/device/devices/local
```

EXPLANATION FOR RETURN CODE

Status code	Description
200	Success.
401	Missing, expired or invalid bearer token.
404	No matching record for the supplied identifier and store.
500	Unexpected server error.

SAMPLE RETURN RESULT

```
{
  "success": true,
  "message": "Devices retrieved successfully.",
  "responsCode": 200,
  "result": [
    {
      "id": 10,
      "mac": "e0000000be65",
      "deviceName": "Aisle 1 - Shelf 2",
      "deviceType": "Minew",
      "screenInch": 4.20,
      "screenColor": "bwry",
      "battery": 100,
      "isOnline": true,
      "storeId": 3
    }
  ]
}
```

3.4 Query template list

URL	https://your-server/api/device/template/local
Request method	GET
Content-type	—
Authorisation	Bearer token.
Description	Label templates synchronised from the Minew console. A template determines what the label draws — price, name, barcode and so on.

- Template identifiers are long numeric **strings**. Keep them quoted in JSON; parsing one as a number loses precision and produces an identifier that does not exist.
- A template belongs to one Minew store and one screen size. Using a template from another store, or one whose size does not match the label, is accepted by SmartShelf and then rejected by the cloud when the label is bound.

REQUEST PARAMETERS

This endpoint takes no parameters.

REQUEST DEMO

```
GET /api/device/template/local
```

EXPLANATION FOR RETURN CODE

Status code	Description
200	Success.
401	Missing, expired or invalid bearer token.
404	No matching record for the supplied identifier and store.
500	Unexpected server error.

SAMPLE RETURN RESULT

```
{
  "success": true,
  "message": "Templates retrieved successfully.",
  "responsCode": 200,
  "result": [
    {
      "id": "2087930462289793024",
      "name": "RIT 4.2 Dynamic",
      "screenInch": 4.20,
      "screenWidth": 400,
      "screenHeight": 300,
      "color": "bwry",
      "storeId": 3,
      "isActive": true
    }
  ]
}
```

Part 4. Product API

Create, amend and retrieve individual products. A product may be created on its own, or together with the labels that display it. Where labels are supplied, the price is pushed and the labels are bound as part of the same call.

4.1 Add product

URL	<code>https://your-server/api/products/product</code>
Request method	POST
Content-type	application/json;charset=utf-8
Authorisation	Bearer token. Admin, Manager or Operator.
Description	Creates a product with no label attached. Use this when the product belongs in the catalogue but has not yet been placed on a shelf.

- Send `subCategoryId: 0` when the product has no sub-category.
- `productCode` must be unique within the store.

REQUEST PARAMETERS

Key	Position	Type	Required	Example	Description
<code>productCode</code>	Body	String	Yes	"PRD-1001"	Unique product code within the store.
<code>productName</code>	Body	String	Yes	"Ceylon Black Tea 200g"	Display name.
<code>barCode</code>	Body	String	No	"4791234500011"	Barcode printed on the label.
<code>categoryId</code>	Body	Long	Yes	3	From Part 3.1.
<code>subCategoryId</code>	Body	Long	No	4	From Part 3.2. Send 0 when not applicable.
<code>quantity</code>	Body	Decimal	No	120	Stock quantity held on the product record.
<code>unitOfMeasure</code>	Body	String	No	"Pack"	Unit shown alongside the quantity.
<code>costPrice</code>	Body	Decimal	No	780.00	Cost price.
<code>sellingPrice</code>	Body	Decimal	Yes	1050.00	Shelf price. This is the value rendered on the label.
<code>discountPrice</code>	Body	Decimal	No	105.00	Discount amount.

Key	Position	Type	Required	Example	Description
discountedPrice	Body	Decimal	No	945.00	Price after discount.
discountPercentage	Body	Decimal	No	10.00	Discount percentage.
wholesalePrice	Body	Decimal	No	900.00	Wholesale price.
minimumPrice	Body	Decimal	No	850.00	Lowest permitted selling price.
maximumPrice	Body	Decimal	No	1200.00	Highest permitted selling price.
description	Body	String	No	"Pure Ceylon tea"	Free-text description.
isActive	Body	Boolean	No	true	Defaults to true.
createdUser	Body	Int	Yes	42	Identifier of the user making the change.
storeId	Body	Long	Yes	3	Store the product belongs to.

REQUEST DEMO

```
{
  "productCode": "PRD-1001",
  "productName": "Ceylon Black Tea 200g",
  "barCode": "4791234500011",
  "categoryId": 3,
  "subCategoryId": 4,
  "quantity": 120,
  "unitOfMeasure": "Pack",
  "costPrice": 780.00,
  "sellingPrice": 1050.00,
  "discountPrice": 105.00,
  "discountedPrice": 945.00,
  "discountPercentage": 10.00,
  "wholesalePrice": 900.00,
  "minimumPrice": 850.00,
  "maximumPrice": 1200.00,
  "description": "Pure Ceylon black tea leaves",
  "isActive": true,
  "createdUser": 42,
  "storeId": 3
}
```

EXPLANATION FOR RETURN CODE

Status code	Description
201	Created.
400	A product with the same code already exists, or the category is invalid.

Status code	Description
403	The user's role does not permit creating products.
500	Unexpected server error.

SAMPLE RETURN RESULT

```
{
  "success": true,
  "message": "Product created successfully.",
  "responsCode": 201,
  "result": {
    "id": 6,
    "productCode": "PRD-1001",
    "productName": "Ceylon Black Tea 200g",
    "sellingPrice": 1050.00,
    "storeId": 3,
    "createdDate": "2026-08-14T09:30:00"
  }
}
```

4.2 Add product with labels

URL	<code>https://your-server/api/products/with-esl</code>
Request method	POST
Content-type	application/json;charset=utf-8
Authorisation	Bearer token. Admin, Manager or Operator.
Description	Creates a product together with the labels that display it, in a single database transaction. If any part fails nothing is saved, so a product cannot be left half-configured with a label pointing at nothing.

- This one call puts the product on the label. The assignment is saved, the price is pushed and the label is bound, so it begins displaying immediately.
- Send `"bindToEsl": false` to record the assignment without contacting the cloud. The label will not display the product until it is bound.
- Send `"eslAssignments": []` to create the product without any label.

REQUEST PARAMETERS

Key	Position	Type	Required	Example	Description
<code>product</code>	Body	Object	Yes	—	Product fields, as in Part 4.1 but without <code>createdUser</code> .

Key	Position	Type	Required	Example	Description
<code>product.storeId</code>	Body	Long	Yes	<code>3</code>	Store the product belongs to.
<code>eslAssignments</code>	Body	Array	No	<code>—</code>	One entry per label. Omit or send an empty array for no label.
<code>deviceId</code>	Body	Long	Yes	<code>10</code>	Label identifier, from Part 3.3.
<code>templateId</code>	Body	String	Yes	<code>"2087930462289793024"</code>	Template identifier, from Part 3.4. Keep quoted.
<code>messageId</code>	Body	Long	No	<code>null</code>	Optional promotional message rendered with the template.
<code>displayOrder</code>	Body	Int	No	<code>1</code>	Order when several labels serve one product.
<code>isActive</code>	Body	Boolean	No	<code>true</code>	Defaults to true.
<code>userId</code>	Body	Int	Yes	<code>42</code>	Identifier of the user making the change.
<code>bindToEsl</code>	Body	Boolean	No	<code>true</code>	Bind the labels in the cloud. Defaults to true.

REQUEST DEMO

```
{
  "product": {
    "productCode": "PRD-1002",
    "productName": "Ground Coffee 250g",
    "barCode": "4791234500028",
    "categoryId": 3,
    "subCategoryId": 4,
    "quantity": 60,
    "unitOfMeasure": "Pack",
    "costPrice": 1200.00,
    "sellingPrice": 1690.00,
    "discountPrice": 90.00,
    "discountedPrice": 1600.00,
    "isActive": true,
    "storeId": 3
  },
  "eslAssignments": [
    {
      "deviceId": 10,
      "templateId": "2087930462289793024",
      "messageId": null,
      "displayOrder": 1,
      "isActive": true
    }
  ],
  "userId": 42,
  "bindToEsl": true
}
```

EXPLANATION FOR RETURN CODE

Status code	Description
201	Created. The message states how many labels were bound.
400	Duplicate product code, invalid category, or an unknown label or template.
403	The user's role does not permit creating products.
500	Unexpected server error. Nothing was saved.

SAMPLE RETURN RESULT

```
{
  "success": true,
  "message": "Product and ESL assignments created successfully. 1 label(s) bound.",
  "responsCode": 201,
  "result": {
    "id": 7,
    "productCode": "PRD-1002",
    "productName": "Ground Coffee 250g",
    "sellingPrice": 1690.00,
    "storeId": 3,
    "createdDate": "2026-08-14T09:35:00"
  }
}
```

4.3 Modify product

URL	<code>https://your-server/api/products/product/{id}</code>
Request method	PUT
Content-type	application/json;charset=utf-8
Authorisation	Bearer token. Admin, Manager or Operator.
Description	Replaces the product's fields and pushes the new price to any label already bound to it.

- This endpoint **replaces** the record. Any field omitted from the request is overwritten with its default. For a price-only feed use **Part 5.2**, which changes only the fields supplied.

REQUEST PARAMETERS

Key	Position	Type	Required	Example	Description
<code>id</code>	Path	Long	Yes	<code>6</code>	Product identifier.
<code>...</code>	Body	—	—	<code>—</code>	Same fields as Part 4.1, with <code>updatedUser</code> in place of <code>createdUser</code> .

REQUEST DEMO

```
{
  "productCode": "PRD-1001",
  "productName": "Ceylon Black Tea 200g",
  "barCode": "4791234500011",
  "categoryId": 3,
  "subCategoryId": 4,
  "quantity": 118,
  "unitOfMeasure": "Pack",
  "costPrice": 780.00,
  "sellingPrice": 1099.00,
  "discountPrice": 99.00,
  "discountedPrice": 1000.00,
  "description": "Pure Ceylon black tea leaves",
  "isActive": true,
  "updatedUser": 42,
  "storeId": 3
}
```

EXPLANATION FOR RETURN CODE

Status code	Description
<code>200</code>	Success.
<code>400</code>	The request was rejected. <code>message</code> carries the reason.

Status code	Description
401	Missing, expired or invalid bearer token.
403	The signed-in user's role does not permit this operation.
404	No matching record.
500	Unexpected server error. <code>error</code> carries the exception text.

SAMPLE RETURN RESULT

```
{
  "success": true,
  "message": "Product updated successfully.",
  "responseCode": 200,
  "result": {
    "id": 6,
    "productCode": "PRD-1001",
    "sellingPrice": 1099.00,
    "updatedAt": "2026-08-14T10:02:00"
  }
}
```

4.4 Modify product with labels

URL	<code>https://your-server/api/products/{id}/with-esl</code>
Request method	PUT
Content-type	application/json;charset=utf-8
Authorisation	Bearer token. Admin, Manager or Operator.
Description	Updates the product and reconciles the labels bound to it, in a single transaction. The request body is the same shape as Part 4.2 .

- To **remove** a label, send its row with `"isDeleted": true` together with the `templateAssignmentId` returned by **Part 4.6**. Removing an assignment also unbinds the label in the cloud, so it stops showing the old price.
- Labels present in the request that are not already bound are bound as part of this call.

REQUEST PARAMETERS

Key	Position	Type	Required	Example	Description
<code>id</code>	Path	Long	Yes	7	Product identifier.
<code>product</code>	Body	Object	Yes	–	Product fields, as in Part 4.2.

Key	Position	Type	Required	Example	Description
<code>eslAssignments</code>	Body	Array	No	–	Desired label state after the call.
<code>templateAssignmentId</code>	Body	Long	No	9	Identifier of an existing binding, required when removing it.
<code>isDeleted</code>	Body	Boolean	No	<code>false</code>	Set true to unbind this label.
<code>userId</code>	Body	Int	Yes	42	Identifier of the user making the change.

REQUEST DEMO

```
{
  "product": {
    "productCode": "PRD-1002",
    "productName": "Ground Coffee 250g",
    "categoryId": 3,
    "sellingPrice": 1750.00,
    "discountPrice": 150.00,
    "isActive": true,
    "storeId": 3
  },
  "eslAssignments": [
    {
      "deviceId": 10,
      "templateId": "2087930462289793024",
      "templateAssignmentId": 9,
      "displayOrder": 1,
      "isActive": true,
      "isDeleted": false
    }
  ],
  "userId": 42
}
```

EXPLANATION FOR RETURN CODE

Status code	Description
200	Success.
400	The request was rejected. <code>message</code> carries the reason.
401	Missing, expired or invalid bearer token.
403	The signed-in user's role does not permit this operation.
404	No matching record.
500	Unexpected server error. <code>error</code> carries the exception text.

SAMPLE RETURN RESULT

```
{
  "success": true,
  "message": "Product and ESL assignments updated successfully. 1 label(s) bound.",
  "responsCode": 200,
  "result": {
    "id": 7,
    "productCode": "PRD-1002",
    "sellingPrice": 1750.00,
    "updatedAt": "2026-08-14T10:20:00"
  }
}
```

4.5 Query product list

URL	<code>https://your-server/api/products</code>
Request method	GET
Content-type	—
Authorisation	Bearer token.
Description	Paged product list for one store. Returns product fields only, without label information. Use Part 4.6 or Part 4.7 when label detail is needed.

- When nothing matches, the response is 404 with `totalCount: 0` rather than an empty 200.

REQUEST PARAMETERS

Key	Position	Type	Required	Example	Description
<code>storeId</code>	Query	Long	Yes	3	Store to list products for.
<code>pageNumber</code>	Query	Int	No	1	Page number. Defaults to 1.
<code>pageSize</code>	Query	Int	No	20	Rows per page. Defaults to 10.
<code>categoryId</code>	Query	Long	No	3	Restricts to one category.
<code>subcategoryId</code>	Query	Long	No	4	Restricts to one sub-category.
<code>searchTerm</code>	Query	String	No	"tea"	Matches product code, name and barcode.

REQUEST DEMO

```
GET /api/products?storeId=3&pageNumber=1&pageSize=20&searchTerm=tea
```

EXPLANATION FOR RETURN CODE

Status code	Description
200	Success.
401	Missing, expired or invalid bearer token.
404	No products matched.
500	Unexpected server error.

SAMPLE RETURN RESULT

```
{
  "success": true,
  "message": "Products retrieved successfully.",
  "responsCode": 200,
  "result": {
    "items": [
      {
        "id": 6,
        "productCode": "PRD-1001",
        "productName": "Ceylon Black Tea 200g",
        "categoryName": "Beverages",
        "subCategoryName": "Tea & Coffee",
        "sellingPrice": 1050.00,
        "discountPrice": 105.00,
        "isActive": true,
        "storeId": 3
      }
    ],
    "totalCount": 1,
    "pageNumber": 1,
    "pageSize": 20,
    "totalPages": 1,
    "searchTerm": "tea"
  }
}
```

4.6 Query product by product code

URL	<code>https://your-server/api/products/by-code/{productCode}</code>
Request method	GET
Content-type	—
Authorisation	Bearer token.
Description	The complete product record for a product code, including every label bound to it . A single call answers what the product is and which labels are showing it.

- `storeId` is required because product codes are unique per store.
- `hasEsl` allows a caller to branch without inspecting the array.
- A label bound with both a template and a message appears **once**, with both sets of fields populated.
- `templateAssignmentId` and `messageAssignmentId` are the values sent back to **Part 4.4** to unbind a label.

REQUEST PARAMETERS

Key	Position	Type	Required	Example	Description
<code>productCode</code>	Path	String	Yes	<code>"PRD-1001"</code>	Product code to look up.
<code>storeId</code>	Query	Long	Yes	<code>3</code>	Store the product belongs to.

REQUEST DEMO

```
GET /api/products/by-code/PRD-1001?storeId=3
```

EXPLANATION FOR RETURN CODE

Status code	Description
<code>200</code>	Success.
<code>401</code>	Missing, expired or invalid bearer token.
<code>404</code>	No matching record for the supplied identifier and store.
<code>500</code>	Unexpected server error.

SAMPLE RETURN RESULT

```
{
  "success": true,
  "message": "Product retrieved successfully.",
  "responsCode": 200,
  "result": {
    "id": 6,
    "productCode": "PRD-1001",
    "barCode": "4791234500011",
    "productName": "Ceylon Black Tea 200g",
    "categoryId": 3,
    "categoryName": "Beverages",
    "subCategoryId": 4,
    "subCategoryName": "Tea & Coffee",
    "quantity": 120.0,
    "unitOfMeasure": "Pack",
    "costPrice": 780.00,
    "sellingPrice": 1050.00,
    "discountPrice": 105.00,
    "discountedPrice": 945.00,
    "isActive": true,
    "storeId": 3,
    "storeName": "Colombo City Centre",
    "hasEsl": true,
    "eslDevices": [
      {
        "deviceId": 10,
        "mac": "e0000000be65",
        "deviceName": "Aisle 1 - Shelf 2",
        "deviceType": "Minew",
        "status": "Active",
        "battery": 100,
        "isOnline": true,
        "lastSeen": "2026-08-14T09:40:00",
        "templateId": "2087930462289793024",
        "templateName": "RIT 4.2 Dynamic",
        "templateAssignmentId": 9,
        "deviceTemplateComboId": 6,
        "messageId": null,
        "messageName": null,
        "displayOrder": 1
      }
    ]
  }
}
```

4.7 Query product by identifier

URL	<code>https://your-server/api/products/{id}/details</code>
Request method	GET
Content-type	—
Authorisation	Bearer token.
Description	Identical payload to Part 4.6 , keyed on the internal product identifier instead of the product code.

REQUEST PARAMETERS

Key	Position	Type	Required	Example	Description
id	Path	Long	Yes	6	Product identifier.
storeId	Query	Long	Yes	3	Store the product belongs to.

REQUEST DEMO

```
GET /api/products/6/details?storeId=3
```

EXPLANATION FOR RETURN CODE

Status code	Description
200	Success.
401	Missing, expired or invalid bearer token.
404	No matching record for the supplied identifier and store.
500	Unexpected server error.

SAMPLE RETURN RESULT

As Part 4.6.

4.8 Delete product

URL	<code>https://your-server/api/products/product/{id}</code>
Request method	DELETE
Content-type	—
Authorisation	Bearer token. Admin, Manager or Operator.
Description	Removes a product. Any label bound to it is released so it stops displaying the product.

REQUEST PARAMETERS

Key	Position	Type	Required	Example	Description
id	Path	Long	Yes	6	Product identifier.
storeId	Query	Long	Yes	3	Store the product belongs to.

REQUEST DEMO

```
DELETE /api/products/product/6?storeId=3
```

EXPLANATION FOR RETURN CODE

Status code	Description
200	Success.
400	The request was rejected. <code>message</code> carries the reason.
401	Missing, expired or invalid bearer token.
403	The signed-in user's role does not permit this operation.
404	No matching record.
500	Unexpected server error. <code>error</code> carries the exception text.

SAMPLE RETURN RESULT

```
{
  "success": true,
  "message": "Product deleted successfully.",
  "responsCode": 200,
  "result": true
}
```

Part 5. Bulk API

Load or update many products from a single JSON payload. These are the endpoints an external point-of-sale or ERP system uses. Both are **partial-success**: a row that fails is reported with a reason and the remaining rows still complete.

Reading the result

Both endpoints answer `200` even when some rows fail, so the HTTP status alone is not a reliable indicator. Walk `result.results[]` and match each entry back to your source row using `index`. The envelope's `success` is `true` only when every row succeeded.

Data and display are reported separately

`eslPushed` indicates the product data reached the cloud. `eslBound` indicates the label is displaying it. Either can fail independently, and a row that carries no labels reports `eslBound: null`. Because the database write is committed before the cloud is contacted, a cloud outage appears as `success: true` with `eslPushed: false` rather than losing the price change.

5.1 Add products in batch

URL	<code>https://your-server/api/products/bulk</code>
Request method	POST
Content-type	application/json;charset=utf-8
Authorisation	Bearer token. Admin, Manager or Operator.
Description	Creates many products from one payload, optionally with their labels, and pushes each price in the same call.

- A category may be given as `categoryId` or as `categoryName`, so an external system need not know SmartShelf's internal identifiers. `categoryId` takes precedence when both are supplied.
- Set `"pushToEsl": false` for a data-only load such as a migration, and `"bindToEsl": false` to save assignments without binding the labels.
- A row that fails is rolled back on its own; the rest of the payload is unaffected.

REQUEST PARAMETERS

Key	Position	Type	Required	Example	Description
<code>storeId</code>	Body	Long	Yes	<code>3</code>	Store all rows belong to.
<code>userId</code>	Body	Int	Yes	<code>42</code>	Identifier of the user making the change.
<code>pushToEsl</code>	Body	Boolean	No	<code>true</code>	Push prices to the cloud. Defaults to true.

Key	Position	Type	Required	Example	Description
<code>bindToEsl</code>	Body	Boolean	No	<code>true</code>	Bind supplied labels. Defaults to true.
<code>products</code>	Body	Array	Yes	<code>–</code>	Rows to create. Must contain at least one entry.
<code>productCode</code>	Body	String	Yes	<code>"BULK-0001"</code>	Unique within the store.
<code>productName</code>	Body	String	Yes	<code>"Bulk Item One"</code>	Display name.
<code>categoryId</code>	Body	Long	No*	<code>3</code>	* Either this or <code>categoryName</code> is required.
<code>categoryName</code>	Body	String	No*	<code>"Beverages"</code>	Matched exactly against active categories in the store.
<code>sellingPrice</code>	Body	Decimal	Yes	<code>450.00</code>	Shelf price.
<code>eslAssignments</code>	Body	Array	No	<code>–</code>	Labels to bind, as in Part 4.2.

REQUEST DEMO

```
{
  "storeId": 3,
  "userId": 42,
  "pushToEsl": true,
  "bindToEsl": true,
  "products": [
    {
      "productCode": "BULK-0001",
      "productName": "Bulk Item One",
      "barCode": "4791234510001",
      "categoryName": "Beverages",
      "quantity": 100,
      "unitOfMeasure": "Pack",
      "costPrice": 300.00,
      "sellingPrice": 450.00,
      "discountPrice": 25.00,
      "isActive": true
    },
    {
      "productCode": "BULK-0002",
      "productName": "Bulk Item Two",
      "categoryId": 3,
      "sellingPrice": 890.00,
      "isActive": true,
      "eslAssignments": [
        { "deviceId": 10, "templateId": "2087930462289793024", "displayOrder": 1 }
      ]
    }
  ]
}
```

EXPLANATION FOR RETURN CODE

Status code	Description
200	The batch was processed. Individual rows may still have failed.
400	The payload was empty, or <code>storeId</code> was missing or unknown.
403	The user's role does not permit creating products.
500	Unexpected server error before the batch began.

SAMPLE RETURN RESULT

```
{
  "success": false,
  "message": "Bulk create finished: 2 created, 1 failed.",
  "responsCode": 200,
  "result": {
    "storeId": 3,
    "totalRows": 3,
    "succeeded": 2,
    "failed": 1,
    "eslPushSucceeded": 2,
    "eslPushFailed": 0,
    "eslBoundSucceeded": 1,
    "eslBoundFailed": 0,
    "eslSummary": "ESL push: 2 succeeded, 0 failed. Label bind: 1 succeeded, 0 failed.",
    "results": [
      { "index": 0, "productCode": "BULK-0001", "productId": 18, "success": true,
        "message": "Created.", "eslPushed": true, "eslMessage": "Pushed to ESL.",
        "eslBound": null, "eslBindMessage": null },
      { "index": 1, "productCode": "BULK-0002", "productId": 19, "success": true,
        "message": "Created.", "eslPushed": true, "eslMessage": "Pushed to ESL.",
        "eslBound": true, "eslBindMessage": "1 label(s) bound." },
      { "index": 2, "productCode": "BULK-0003", "productId": null, "success": false,
        "message": "Category not found (CategoryId=, CategoryName='Unknown').",
        "eslPushed": null, "eslMessage": null, "eslBound": null, "eslBindMessage": null }
    ]
  }
}
```

5.2 Modify products and prices in batch

URL	https://your-server/api/products/bulk
Request method	PUT
Content-type	application/json;charset=utf-8
Authorisation	Bearer token. Admin, Manager or Operator.
Description	Updates many products from one payload and pushes every new price to its label in the same request. This is the endpoint a price feed should use.

- Rows are matched on `productCode` within `storeId`. Send `id` instead to match on the internal identifier.
- **Only the fields supplied are changed.** Every field is optional, so the price-only payload below will not blank descriptions, barcodes or categories. This is the difference from **Part 4.3**, which replaces the whole record.
- Labels already bound continue to display the product; there is no need to re-bind after a price change.

REQUEST PARAMETERS

Key	Position	Type	Required	Example	Description
<code>storeId</code>	Body	Long	Yes	3	Store all rows belong to.
<code>userId</code>	Body	Int	Yes	42	Identifier of the user making the change.
<code>pushToEsl</code>	Body	Boolean	No	true	Push prices to the cloud. Defaults to true.
<code>products</code>	Body	Array	Yes	–	Rows to update.
<code>productCode</code>	Body	String	Yes*	"PRD-1001"	* Either this or <code>id</code> is required.
<code>id</code>	Body	Long	No*	6	Match on the internal identifier instead.
<code>sellingPrice</code>	Body	Decimal	No	1199.50	Any field may be omitted to leave it unchanged.
<code>discountPrice</code>	Body	Decimal	No	99.50	Discount amount.

REQUEST DEMO

```
{
  "storeId": 3,
  "userId": 42,
  "pushToEsl": true,
  "products": [
    { "productCode": "PRD-1001", "sellingPrice": 1199.50, "discountPrice": 99.50 },
    { "productCode": "PRD-1002", "sellingPrice": 1750.00 }
  ]
}
```

EXPLANATION FOR RETURN CODE

Status code	Description
200	The batch was processed. Individual rows may still have failed.
400	The payload was empty, or <code>storeId</code> was missing or unknown.
403	The user's role does not permit updating products.
500	Unexpected server error before the batch began.

SAMPLE RETURN RESULT

```
{
  "success": false,
  "message": "Bulk update finished: 2 updated, 1 failed.",
  "responsCode": 200,
  "result": {
    "storeId": 3,
    "totalRows": 3,
    "succeeded": 2,
    "failed": 1,
    "eslPushSucceeded": 2,
    "eslPushFailed": 0,
    "eslSummary": "ESL push: 2 succeeded, 0 failed.",
    "results": [
      { "index": 0, "productCode": "PRD-1001", "productId": 6, "success": true,
        "message": "Updated.", "eslPushed": true, "eslMessage": "Pushed to ESL." },
      { "index": 1, "productCode": "PRD-1002", "productId": 7, "success": true,
        "message": "Updated.", "eslPushed": true, "eslMessage": "Pushed to ESL." },
      { "index": 2, "productCode": "NOPE-999", "productId": null, "success": false,
        "message": "Product code 'NOPE-999' not found in store 3.",
        "eslPushed": null, "eslMessage": null }
    ]
  }
}
```

Part 6. Label API

Inspect an individual electronic shelf label and, where necessary, force it to redraw. Labels are normally bound as part of creating or updating a product; the endpoints here exist for diagnosis and recovery.

6.1 Query label by MAC address

URL	<code>https://your-server/api/device/device/by-mac/{mac}</code>
Request method	GET
Content-type	—
Authorisation	Bearer token.
Description	The complete record for a label, including what it is currently displaying . This is the first call to make when a label is showing something unexpected.

- The MAC address may be given bare (`e0000000be65`) or separated (`e0:00:00:00:be:65` or `e0-00-00-00-be-65`). Both match the same label, and case is not significant.
- `storeId` is optional and needed only to distinguish a MAC address registered in more than one store.

REQUEST PARAMETERS

Key	Position	Type	Required	Example	Description
<code>mac</code>	Path	String	Yes	<code>"e0000000be65"</code>	Label MAC address, bare or separated.
<code>storeId</code>	Query	Long	No	<code>3</code>	Disambiguates a MAC present in several stores.

REQUEST DEMO

```
GET /api/device/device/by-mac/e0000000be65?storeId=3
```

EXPLANATION FOR RETURN CODE

Status code	Description
<code>200</code>	Success.
<code>401</code>	Missing, expired or invalid bearer token.
<code>404</code>	No matching record for the supplied identifier and store.
<code>500</code>	Unexpected server error.

SAMPLE RETURN RESULT

```
{
  "success": true,
  "message": "Device retrieved successfully.",
  "responsCode": 200,
  "result": {
    "id": 10,
    "mac": "e0000000be65",
    "deviceName": "Aisle 1 - Shelf 2",
    "deviceType": "Minew",
    "minewDeviceId": "2087929882007834624",
    "status": "Active",
    "battery": 100,
    "isOnline": true,
    "isActive": true,
    "lastSeen": "2026-08-14T09:40:00",
    "lastSyncTime": "2026-08-14T09:38:00",
    "firmware": "3.5.0",
    "screenInch": 4.20,
    "screenWidth": 400,
    "screenHeight": 300,
    "screenColor": "bwry",
    "storeId": 3,
    "storeName": "Colombo City Centre",
    "minewStoreId": "2087891877587062784",
    "isBound": true,
    "bindings": [
      {
        "assignmentId": 9,
        "assignmentType": "TEMPLATE",
        "locationType": "Product",
        "locationId": 6,
        "productId": 6,
        "productCode": "PRD-1001",
        "productName": "Ceylon Black Tea 200g",
        "sellingPrice": 1050.00,
        "discountPrice": 105.00,
        "templateId": "2087930462289793024",
        "templateName": "RIT 4.2 Dynamic",
        "messageId": null,
        "messageName": null,
        "displayOrder": 1
      }
    ]
  }
}
```

6.2 Query label by identifier

URL	<code>https://your-server/api/device/device/{id}</code>
Request method	GET
Content-type	—
Authorisation	Bearer token.

Description	Label record by internal identifier. Returns the device fields only; use Part 6.1 for the version that includes what the label is displaying.
--------------------	--

REQUEST PARAMETERS

Key	Position	Type	Required	Example	Description
id	Path	Long	Yes	10	Label identifier.

REQUEST DEMO

GET /api/device/device/10

EXPLANATION FOR RETURN CODE

Status code	Description
200	Success.
401	Missing, expired or invalid bearer token.
404	No matching record for the supplied identifier and store.
500	Unexpected server error.

SAMPLE RETURN RESULT

<pre>{ "success": true, "message": "Device retrieved successfully.", "responsCode": 200, "result": { "id": 10, "mac": "e0000000be65", "deviceName": "Aisle 1 - Shelf 2", "status": "Active", "battery": 100, "isOnline": true, "storeId": 3, "storeName": "Colombo City Centre" } }</pre>

6.3 Redraw label

URL	https://your-server/api/device/bind
Request method	POST
Content-type	application/json;charset=utf-8

Authorisation	Bearer token. Admin or Manager only.
Description	Forces a label to redraw a product it is already assigned to.

- This is not normally required. Creating or updating a product with `eslAssignments` binds the label automatically.
- Use it to recover a label that was asleep or out of range when it was bound, or after a template was changed in the Minew console.
- `comboId` is the `deviceTemplateComboId` returned by **Part 4.6**.

REQUEST PARAMETERS

Key	Position	Type	Required	Example	Description
<code>comboId</code>	Body	Long	Yes	6	Label and template pairing to redraw.
<code>productId</code>	Body	Long	Yes*	6	* Supply this, or an explicit <code>goodsMap</code> .
<code>goodsMap</code>	Body	Object	No*	–	Explicit field values, when not deriving them from a product.

REQUEST DEMO

```
{
  "comboId": 6,
  "productId": 6
}
```

EXPLANATION FOR RETURN CODE

Status code	Description
200	The label was instructed to redraw.
400	The store is not linked to a Minew store, or the cloud rejected the request.
403	The user's role does not permit binding.
404	Unknown pairing or product.
500	Unexpected server error.

SAMPLE RETURN RESULT

```
{
  "message": "Data bound successfully",
  "response": {
    "code": 200,
    "msg": "success",
    "data": null
  }
}
```

Part 7. Scheduling API

Schedule a product onto a label for a defined period — a promotion window, a happy-hour price, a seasonal display. A background service arms each schedule and executes it at the appointed time.

The price is read when the schedule runs

Not when it is created. A promotion scheduled today picks up whatever the selling price is at the moment it executes, so a recurring schedule continues to show the current price without being amended.

Times are UTC

The scheduler compares against UTC. Sri Lanka Standard Time is UTC+5:30, so a promotion intended to begin at 15:00 local time must be sent as `09:30`. This is the most common cause of a schedule that appears never to run.

Execution timing

The service checks for due schedules every 30 seconds and activates at most 20 in each cycle. This is appropriate for promotions and price changes; it is not intended for second-accurate work.

7.1 Add schedule

URL	<code>https://your-server/api/queue/direct</code>
Request method	POST
Content-type	<code>application/json; charset=utf-8</code>
Authorisation	Bearer token. Admin, Manager or Operator.
Description	Schedules a product onto a label between a start and end time.

- `startDate` and `endDate` are **UTC**.
- Always supply `endDate`. The overlap check cannot compare against an open-ended schedule, so omitting it allows conflicting schedules to be created.
- `templateId` is required for Minew labels even when the purpose of the schedule is to show a message.
- `storeId` is not sent; it is taken from the label.

REQUEST PARAMETERS

Key	Position	Type	Required	Example	Description
<code>deviceId</code>	Body	Long	Yes	<code>10</code>	Label to schedule, from Part 3.3.
<code>templateId</code>	Body	String	Yes	<code>"2087930462289793024"</code>	Template to render with. Keep quoted.

Key	Position	Type	Required	Example	Description
messageId	Body	Long	No	null	Optional message rendered alongside the template.
locationType	Body	String	Yes	"PRODUCT"	PRODUCT or SHELF .
locationId	Body	Long	Yes	6	Product identifier when the type is PRODUCT .
startDate	Body	DateTime	Yes	"2026-08-15T09:30:00"	UTC. Format yyyy-MM-ddTHH:mm:ss .
endDate	Body	DateTime	No	"2026-08-15T14:30:00"	UTC. Strongly recommended — see note above.
priorityId	Body	Int	No	2	See Part 7.11. Defaults to 4 (Scheduled).
isRecurring	Body	Boolean	No	false	Repeat according to recurrencePattern .
recurrencePattern	Body	String	No	"DAILY"	DAILY , WEEKLY , MONTHLY or HOURLY .
userId	Body	Int	Yes	42	Identifier of the user making the change.
queueType	Body	String	No	"TEMPLATE_QUEUE"	Defaults by label type.

REQUEST DEMO

```
{
  "deviceId": 10,
  "templateId": "2087930462289793024",
  "messageId": null,
  "locationType": "PRODUCT",
  "locationId": 6,
  "startDate": "2026-08-15T09:30:00",
  "endDate": "2026-08-15T14:30:00",
  "priorityId": 2,
  "isRecurring": false,
  "recurrencePattern": null,
  "userId": 42,
  "queueType": "TEMPLATE_QUEUE"
}
```

EXPLANATION FOR RETURN CODE

Status code	Description
201	Scheduled.

Status code	Description
400	Invalid request. A Minew label was scheduled without a template, or the location does not exist.
409	An unfinished schedule already covers this label and template over an overlapping period.
500	Unexpected server error.

SAMPLE RETURN RESULT

```
{
  "success": true,
  "message": "Queue created successfully",
  "responsCode": 201,
  "result": {
    "id": 13,
    "queueType": "TEMPLATE_QUEUE",
    "deviceName": "Aisle 1 - Shelf 2",
    "deviceType": "Minew",
    "locationType": "PRODUCT",
    "locationName": "Ceylon Black Tea 200g",
    "productId": 6,
    "productName": "Ceylon Black Tea 200g",
    "startDate": "2026-08-15T09:30:00Z",
    "endDate": "2026-08-15T14:30:00Z",
    "isActive": true,
    "isRecurring": false,
    "displayOrder": 1,
    "storeId": 3
  }
}
```

7.2 Add schedule from an existing binding

URL	<code>https://your-server/api/queue/from-assignment</code>
Request method	POST
Content-type	application/json;charset=utf-8
Authorisation	Bearer token. Admin, Manager or Operator.
Description	Schedules a label that is already bound, so the label, template and product need not be repeated.

- `assignmentId` is the `templateAssignmentId` returned by **Part 4.6**.

REQUEST PARAMETERS

Key	Position	Type	Required	Example	Description
<code>assignmentId</code>	Body	Long	Yes	9	Existing binding to schedule.

Key	Position	Type	Required	Example	Description
startDate	Body	DateTime	Yes	"2026-08-15T09:30:00"	UTC.
endDate	Body	DateTime	No	"2026-08-15T14:30:00"	UTC.
priorityId	Body	Int	No	2	See Part 7.11.
displayOrder	Body	Int	No	1	Order among several labels.
isRecurring	Body	Boolean	No	false	Repeat the schedule.
recurrencePattern	Body	String	No	null	Recurrence interval.
userId	Body	Int	Yes	42	Identifier of the user making the change.

REQUEST DEMO

```
{
  "assignmentId": 9,
  "startDate": "2026-08-15T09:30:00",
  "endDate": "2026-08-15T14:30:00",
  "priorityId": 2,
  "displayOrder": 1,
  "isRecurring": false,
  "recurrencePattern": null,
  "userId": 42
}
```

EXPLANATION FOR RETURN CODE

Status code	Description
201	Scheduled.
400	Invalid request.
404	Unknown binding.
409	An overlapping schedule already exists.
500	Unexpected server error.

SAMPLE RETURN RESULT

As Part 7.1.

7.3 Query schedule list

URL	<code>https://your-server/api/queue</code>
Request method	GET
Content-type	—
Authorisation	Bearer token.
Description	Paged list of schedules, with filtering.

REQUEST PARAMETERS

Key	Position	Type	Required	Example	Description
<code>storeId</code>	Query	Long	No	<code>3</code>	Restricts to one store.
<code>deviceId</code>	Query	Long	No	<code>10</code>	Restricts to one label.
<code>status</code>	Query	String	No	<code>"Pending"</code>	<code>Pending</code> , <code>Processing</code> , <code>Completed</code> or <code>Failed</code> .
<code>locationType</code>	Query	String	No	<code>"PRODUCT"</code>	Restricts by location type.
<code>isActive</code>	Query	Boolean	No	<code>true</code>	Restricts to live or retired schedules.
<code>pageNumber</code>	Query	Int	No	<code>1</code>	Page number.
<code>pageSize</code>	Query	Int	No	<code>20</code>	Rows per page.

REQUEST DEMO

```
GET /api/queue?storeId=3&status=Pending&pageNumber=1&pageSize=20
```

EXPLANATION FOR RETURN CODE

Status code	Description
<code>200</code>	Success.
<code>401</code>	Missing, expired or invalid bearer token.
<code>404</code>	No matching record for the supplied identifier and store.
<code>500</code>	Unexpected server error.

SAMPLE RETURN RESULT

```
{
  "success": true,
  "message": "Queues retrieved successfully",
  "responsCode": 200,
  "result": {
    "items": [
      {
        "id": 13,
        "queueType": "TEMPLATE_QUEUE",
        "deviceName": "Aisle 1 - Shelf 2",
        "locationType": "PRODUCT",
        "locationName": "Ceylon Black Tea 200g",
        "startDate": "2026-08-15T09:30:00Z",
        "endDate": "2026-08-15T14:30:00Z",
        "status": "Pending",
        "priority": "Price Change",
        "isActive": true
      }
    ],
    "totalCount": 1,
    "pageNumber": 1,
    "pageSize": 20,
    "totalPages": 1
  }
}
```

7.4 Query schedule and execution history

URL	<code>https://your-server/api/queue/{id}</code>
Request method	GET
Content-type	—
Authorisation	Bearer token.
Description	One schedule together with a record of every attempt to execute it.

- When a label did not change, this is where the reason is. `errorMessage` carries the cloud's own rejection text and `bindingData` holds its raw response.
- A rejected instruction is recorded as **Failed** rather than silently marked complete, so a status of Failed always reflects a genuine attempt.

REQUEST PARAMETERS

Key	Position	Type	Required	Example	Description
<code>id</code>	Path	Long	Yes	<code>13</code>	Schedule identifier.

REQUEST DEMO

GET /api/queue/13

EXPLANATION FOR RETURN CODE

Status code	Description
200	Success.
401	Missing, expired or invalid bearer token.
404	No matching record for the supplied identifier and store.
500	Unexpected server error.

SAMPLE RETURN RESULT

```
{
  "success": true,
  "message": "Queue retrieved successfully",
  "responseCode": 200,
  "result": {
    "id": 13,
    "deviceName": "Aisle 1 - Shelf 2",
    "productName": "Ceylon Black Tea 200g",
    "startDate": "2026-08-15T09:30:00Z",
    "endDate": "2026-08-15T14:30:00Z",
    "status": "Completed",
    "priority": "Price Change",
    "lastAttempt": "2026-08-15T09:30:12Z",
    "retryCount": 0,
    "errorMessage": null,
    "bindingData": "{ \"code\": 200, \"msg\": \"success\", \"data\": null }"
  }
}
```

7.5 Query upcoming schedules

URL	https://your-server/api/queue/upcoming
Request method	GET
Content-type	—
Authorisation	Bearer token.
Description	Schedules armed to run within the next given number of hours. The quickest check that a schedule was accepted and will execute.

REQUEST PARAMETERS

Key	Position	Type	Required	Example	Description
hours	Query	Int	No	24	Look-ahead window. Defaults to 24.

REQUEST DEMO

```
GET /api/queue/upcoming?hours=24
```

EXPLANATION FOR RETURN CODE

Status code	Description
200	Success.
401	Missing, expired or invalid bearer token.
404	No matching record for the supplied identifier and store.
500	Unexpected server error.

SAMPLE RETURN RESULT

```
{
  "success": true,
  "message": "Upcoming queues retrieved successfully",
  "responseCode": 200,
  "result": [
    {
      "id": 13,
      "locationType": "PRODUCT",
      "locationName": "Ceylon Black Tea 200g",
      "productId": 6,
      "productName": "Ceylon Black Tea 200g",
      "deviceName": "Aisle 1 - Shelf 2",
      "startDate": "2026-08-15T09:30:00Z",
      "status": "Pending"
    }
  ]
}
```

7.6 Query active schedules

URL	https://your-server/api/queue/active
Request method	GET
Content-type	—
Authorisation	Bearer token.

Description	Schedules currently within their display period.
--------------------	--

REQUEST PARAMETERS

This endpoint takes no parameters.

REQUEST DEMO

```
GET /api/queue/active
```

EXPLANATION FOR RETURN CODE

Status code	Description
200	Success.
401	Missing, expired or invalid bearer token.
404	No matching record for the supplied identifier and store.
500	Unexpected server error.

SAMPLE RETURN RESULT

Same shape as Part 7.5.

7.7 Query schedules for a label

URL	<code>https://your-server/api/queue/device/{deviceId}</code>
Request method	GET
Content-type	—
Authorisation	Bearer token.
Description	Every schedule targeting one label. Useful when a label is showing something unexpected and the cause may be a schedule rather than a direct binding.

REQUEST PARAMETERS

Key	Position	Type	Required	Example	Description
<code>deviceId</code>	Path	Long	Yes	10	Label identifier.

REQUEST DEMO

```
GET /api/queue/device/10
```

EXPLANATION FOR RETURN CODE

Status code	Description
200	Success.
401	Missing, expired or invalid bearer token.
404	No matching record for the supplied identifier and store.
500	Unexpected server error.

SAMPLE RETURN RESULT

Same shape as Part 7.5.

7.8 Modify schedule

URL	<code>https://your-server/api/queue/{id}</code>
Request method	PUT
Content-type	application/json;charset=utf-8
Authorisation	Bearer token. Admin, Manager or Operator.
Description	Moves the display period or changes the recurrence. Only the fields supplied are applied; all are optional except <code>userId</code> .

REQUEST PARAMETERS

Key	Position	Type	Required	Example	Description
<code>id</code>	Path	Long	Yes	13	Schedule identifier.
<code>startDate</code>	Body	DateTime	No	"2026-08-16T09:30:00"	UTC.
<code>endDate</code>	Body	DateTime	No	"2026-08-16T14:30:00"	UTC.
<code>priorityId</code>	Body	Int	No	2	See Part 7.11.
<code>isActive</code>	Body	Boolean	No	true	Retire or reinstate the schedule.
<code>isRecurring</code>	Body	Boolean	No	false	Repeat the schedule.
<code>recurrencePattern</code>	Body	String	No	null	Recurrence interval.
<code>userId</code>	Body	Int	Yes	42	Identifier of the user making the change.

REQUEST DEMO

```
{
  "startDate": "2026-08-16T09:30:00",
  "endDate": "2026-08-16T14:30:00",
  "priorityId": 2,
  "isActive": true,
  "isRecurring": false,
  "recurrencePattern": null,
  "userId": 42
}
```

EXPLANATION FOR RETURN CODE

Status code	Description
200	Success.
400	The request was rejected. <code>message</code> carries the reason.
401	Missing, expired or invalid bearer token.
403	The signed-in user's role does not permit this operation.
404	No matching record.
500	Unexpected server error. <code>error</code> carries the exception text.

SAMPLE RETURN RESULT

```
{
  "success": true,
  "message": "Queue updated successfully",
  "responseCode": 200,
  "result": {
    "id": 13,
    "startDate": "2026-08-16T09:30:00Z",
    "endDate": "2026-08-16T14:30:00Z",
    "status": "Pending"
  }
}
```

7.9 Run schedule immediately

URL	<code>https://your-server/api/queue/{id}/activate</code>
Request method	POST
Content-type	—
Authorisation	Bearer token. Admin, Manager or Operator.

Description	Executes a schedule at once rather than waiting for its start time. The label is instructed to redraw immediately.
--------------------	--

- Do not use this on a schedule whose start time has already passed. The background service will have executed it within 30 seconds, and this call then returns `409`.
- Neither `409` nor `400` alters the schedule's recorded outcome, so a status of Failed always reflects a genuine execution attempt.

REQUEST PARAMETERS

Key	Position	Type	Required	Example	Description
<code>id</code>	Path	Long	Yes	<code>13</code>	Schedule identifier.

REQUEST DEMO

```
POST /api/queue/13/activate
```

EXPLANATION FOR RETURN CODE

Status code	Description
<code>200</code>	Executed. The label was instructed to redraw.
<code>400</code>	The schedule's start time has not yet arrived.
<code>404</code>	Unknown schedule.
<code>409</code>	The schedule has already run.
<code>500</code>	Execution failed. The schedule is recorded as Failed with the reason.

SAMPLE RETURN RESULT

```
{
  "success": true,
  "message": "Queue activated successfully",
  "responseCode": 200,
  "result": {
    "id": 13,
    "status": "Completed",
    "lastAttempt": "2026-08-14T11:05:00Z"
  }
}
```

7.10 Stop schedule

URL	<code>https://your-server/api/queue/{id}/deactivate</code>
------------	--

Request method	POST
Content-type	—
Authorisation	Bearer token. Admin, Manager or Operator.
Description	Retires a running schedule and restores the label's previous display.

- The schedule remains **Completed** rather than reverting to Pending, so the history stays accurate. Its `isActive` flag is cleared, which is also what makes it eligible for deletion.

REQUEST PARAMETERS

Key	Position	Type	Required	Example	Description
<code>id</code>	Path	Long	Yes	<code>13</code>	Schedule identifier.

REQUEST DEMO

```
POST /api/queue/13/deactivate
```

EXPLANATION FOR RETURN CODE

Status code	Description
<code>200</code>	Success.
<code>400</code>	The request was rejected. <code>message</code> carries the reason.
<code>401</code>	Missing, expired or invalid bearer token.
<code>403</code>	The signed-in user's role does not permit this operation.
<code>404</code>	No matching record.
<code>500</code>	Unexpected server error. <code>error</code> carries the exception text.

SAMPLE RETURN RESULT

```
{
  "success": true,
  "message": "Queue deactivated successfully",
  "responsCode": 200,
  "result": {
    "id": 13,
    "status": "Completed",
    "isActive": false
  }
}
```

7.11 Delete schedule

URL	<code>https://your-server/api/queue/{id}</code>
Request method	DELETE
Content-type	—
Authorisation	Bearer token. Admin, Manager or Operator.
Description	Removes a schedule.

- A schedule still within its display period cannot be deleted. Stop it first using **Part 7.10**, then delete it.

REQUEST PARAMETERS

Key	Position	Type	Required	Example	Description
<code>id</code>	Path	Long	Yes	<code>13</code>	Schedule identifier.

REQUEST DEMO

```
DELETE /api/queue/13
```

EXPLANATION FOR RETURN CODE

Status code	Description
<code>200</code>	Deleted.
<code>404</code>	Unknown schedule.
<code>409</code>	The schedule is still displaying on its label. Stop it first.
<code>500</code>	Unexpected server error.

SAMPLE RETURN RESULT

```
{
  "success": true,
  "message": "Queue deleted successfully",
  "responseCode": 200
}
```

7.12 Query priority list

URL	<code>https://your-server/api/queue/prioritytypes</code>
-----	--

Request method	GET
Content-type	—
Authorisation	Bearer token.
Description	The values accepted by <code>priorityId</code> . Priority determines which schedule wins when more than one targets the same label.

REQUEST PARAMETERS

This endpoint takes no parameters.

REQUEST DEMO

```
GET /api/queue/prioritytypes
```

EXPLANATION FOR RETURN CODE

Status code	Description
200	Success.
401	Missing, expired or invalid bearer token.
404	No matching record for the supplied identifier and store.
500	Unexpected server error.

SAMPLE RETURN RESULT

```
{
  "success": true,
  "message": "Priorities retrieved successfully",
  "responseCode": 200,
  "result": [
    { "id": 1, "priorityCode": "EMERGENCY", "priorityName": "Emergency", "value": 1 },
    { "id": 2, "priorityCode": "PRICE_CHANGE", "priorityName": "Price Change", "value": 2 },
    { "id": 3, "priorityCode": "PROMOTION", "priorityName": "Promotion", "value": 3 },
    { "id": 4, "priorityCode": "SCHEDULED", "priorityName": "Scheduled", "value": 4 },
    { "id": 5, "priorityCode": "MAINTENANCE", "priorityName": "Maintenance", "value": 5 }
  ]
}
```

Appendix A. Response envelope

Every endpoint in this document returns the same wrapper. Read `success` first; the payload is always under `result`.

```
{
  "success": true,
  "message": "Product retrieved successfully.",
  "responseCode": 200,
  "error": null,
  "result": { }
}
```

Field	Type	Description
success	Boolean	True when the operation completed as requested. For bulk endpoints this is true only when every row succeeded.
message	String	Human-readable outcome. On failure this carries the reason.
responseCode	Int	Mirrors the HTTP status code.
error	String	Exception text on an unexpected server error; otherwise null.
result	Object	The payload. Shape varies by endpoint.

Appendix B. Status codes

Codes used across the API. Endpoint-specific meanings are listed with each endpoint.

Code	Meaning	Typical cause
200	Success	The request completed. For bulk endpoints, inspect the per-row results.
201	Created	A new product or schedule was created.
400	Bad request	A required field was missing, or a referenced record does not exist.
401	Unauthorised	No token, an expired token, or the email address was sent instead of the Employee ID.
403	Forbidden	The signed-in user holds the Viewer role, or the operation requires Admin or Manager.
404	Not found	No record matched. For product lookups, check that storeId is correct.
409	Conflict	An overlapping schedule exists, a schedule has already run, or a schedule is still displaying.
500	Server error	Unexpected fault. <code>error</code> carries the detail.

Appendix C. Troubleshooting

Conditions encountered most often during integration, and how to resolve them.

Symptom	Cause	Resolution
401 on log in	The email address was sent as <code>userName</code> .	Send the Employee ID, for example <code>ADMIN01</code> .
404 on a product that exists	<code>storeId</code> is missing or refers to another store.	Product codes are unique per store. Supply the correct <code>storeId</code> .
"Category not found"	The category name does not match an active category in that store.	Check against Part 3.1. Matching is exact.
"Product with same code already exists"	Bulk <i>create</i> was used for a product already present.	Use Part 5.2 for products that already exist.
"Store has no <code>MinewStoreId</code> "	The store is not linked to a Minew cloud store.	Link the store once, then resend the affected rows. Until then products save to the database only.
Price accepted but the label did not change	No label is bound to the product.	Supply <code>eslAssignments</code> when creating or updating. Confirm with Part 4.6 — <code>hasEs1</code> should be true.
<code>eslBound: false</code> on a row that saved	The cloud rejected the binding.	Usually a template belonging to another store, or one whose screen size does not match the label. Correct it and resend, or use Part 6.3 .
A template identifier appears rounded	It was parsed as a number and lost precision.	Template identifiers are strings. Keep them quoted throughout.
A schedule never runs	<code>startDate</code> was sent in local time.	Convert to UTC by subtracting 5 hours 30 minutes. Verify with Part 7.5 .
409 when scheduling	An unfinished schedule already covers that label, template and period.	Inspect existing schedules with Part 7.7 , then amend or delete the conflicting one.
409 when deleting a schedule	It is still within its display period.	Stop it with Part 7.10 , then delete.